

5.2 TTS request flow

- Frontend submits text, language, and optional voice ID.
- FastAPI validates input and checks the capability registry.
- TTS service loads or reuses the selected model and generates audio.
- Backend returns audio or a generated-file URL plus metadata.
- Frontend plays the audio and offers a download.

5.3 STT request flow

- Frontend uploads a recording or audio file.
- Backend validates the payload and decodes audio into the model's expected format.
- STT model returns transcript and optional timestamps/language metadata.
- Backend returns text as JSON; frontend allows copying or translating it.
- Temporary uploaded files are removed according to configured cleanup rules.

5.4 Translate-and-speak flow

- User supplies source text and source/target languages.
- Translation model returns target text without replacing the source.
- User reviews the translation and selects Speak.
- TTS receives the translated text and the target-language voice.
- The UI labels the output with its language so the user can distinguish original and translated speech.

5.5 Resource management

Speech models can consume significant RAM and disk space. Load models lazily, reuse loaded models, and avoid loading several large models simultaneously on a memory-constrained Mac. Provide a model-unavailable or insufficient-memory error rather than crashing the entire API. Model quantization or smaller variants may help, but quality must be tested after any optimization.

6. Model Strategy and Candidate Technologies

The project should choose models by task and language coverage, then benchmark them on the actual Mac. A model's repository being open source does not automatically mean its weights, training data, or commercial use are unrestricted.

Task	Candidate direction	Evaluation points
Tamil/English TTS	AI4Bharat Indic-TTS or another model explicitly supporting the target languages	Tamil intelligibility, voice naturalness, latency, memory, license
Tamil/English STT	Whisper/faster-whisper or an Indian-language speech model	Tamil-script transcript quality, code-switching, noise, CPU speed
Tamil ↔ English translation	AI4Bharat IndicTrans2 or compatible translation model	Meaning preservation, names/numbers, idioms, local installation
Optional text preparation	Ollama with a suitably sized local instruction model	Memory use, Tamil handling, no unrequested translation
Voice cloning	A cloning-capable model whose base TTS supports Tamil	Tamil support, consent controls, similarity, license, latency

6.1 Candidate project links

- AI4Bharat Indic-TTS: <https://github.com/AI4Bharat/Indic-TTS>
- AI4Bharat model portal: <https://models.ai4bharat.org/>
- AI4Bharat IndicTrans2: <https://github.com/AI4Bharat/IndicTrans2>
- faster-whisper: <https://github.com/SYSTRAN/faster-whisper>
- Ollama: <https://ollama.com/>
- OpenVoice (evaluate Tamil compatibility before use): <https://github.com/myshell-ai/OpenVoice>

These are candidates, not a claim that they have been installed or benchmarked together. Before implementation, verify current repository status, language coverage, runtime requirements, model-weight terms, dependencies, and commercial-use rights. In particular, do not assume a voice-cloning model supports Tamil just because its cloning feature works for other languages.

6.2 Model registry

```
{
  "task": "tts",
  "language": "ta",
  "model_id": "indic_tts_ta_candidate",
  "enabled": false,
  "verified_on_device": false,
  "license_reviewed": false,
  "notes": "Enable only after local smoke test and license review."
}
```

Maintain a small registry like this in configuration. Enable a language/task pair only after installation, inference, and quality checks pass.

7. Tamil Voice Quality and Voice Cloning

7.1 Tamil voice-quality checklist

- Pronunciation of common Tamil words and longer sentences.
- Correct handling of punctuation, pauses, questions, and exclamations.
- Formal Tamil versus conversational Tamil.
- Names, place names, abbreviations, numbers, dates, and currency.
- Tamil text containing English product names or technical terms.
- Natural speed and intelligibility at normal playback volume.
- Consistency across repeated generations.

7.2 Voice cloning is a separate phase

Voice cloning is not the same as choosing a Tamil voice. A model may clone a speaker in one supported language but fail to produce natural Tamil. The project must verify both speaker similarity and Tamil pronunciation on the chosen base model before exposing the feature as supported.

- Use only recordings from the user or a speaker who has explicitly authorized cloning.
- Record consent, intended use, and permitted retention period in the product workflow.
- Do not allow users to impersonate another person without authorization.
- Keep reference recordings private and provide deletion controls.
- Test cloned voice output with a fixed Tamil evaluation set and compare against a standard voice.
- Label generated audio as AI-generated where appropriate for the use case.

7.3 Do we need to train from scratch?

No for the MVP. Start with pretrained models. If Tamil quality is inadequate, consider fine-tuning or adapting an appropriate model using licensed, consented, well-recorded data. Training from scratch would require substantially more data, compute, time, and expertise than a three-day MVP allows.

8. API Contract

Use FastAPI and Pydantic for validation. Keep endpoints under /api/v1 and generate OpenAPI documentation automatically.

Method & endpoint	Purpose	Request / response
GET /health	Application health	App status, version
GET /health/models	Model readiness	Per-task and per-language readiness
POST /api/v1/tts	Text to speech	JSON request; audio response or job/file reference
POST /api/v1/stt	Speech to text	Multipart audio + language; transcript JSON
POST /api/v1/translate	Translation	Source text + source/target language; translated text
POST /api/v1/text/prepare	Optional text preparation	Text + raw/enhanced/expressive mode
GET /api/v1/voices	List enabled voices	Voice IDs, supported languages, labels
POST /api/v1/voices/clone	Later: create voice profile	Consent-gated reference audio and metadata
DELETE /api/v1/voices/{id}	Later: delete voice profile	Confirmation and deletion result

8.1 Example TTS request

```
POST /api/v1/tts
Content-Type: application/json
```

```
{
  "text": "■■■■■■■! ■■■■ ■■■■■■ ■■■■■■ ■■■■■■■■■■■■?",
  "language": "ta",
  "voice_id": "ta_default",
  "format": "wav"
}
```

8.2 Example STT response

```
{
  "text": "■■■■■■■■■, ■■■■■■■ ■■■■■ ■■■■■■■■■■■■■■■■■■■■■?",
  "language": "ta",
  "model_id": "stt_candidate",
  "duration_seconds": 4.8
}
```

8.3 Example translation request

POST /api/v1/translate

```
{
  "text": "■■■■■■■, ■■■■■■ ■■■■■ ■■■■■■■■■■■■■?",
  "source_language": "ta",
  "target_language": "en"
}
```

Example response shape

```
{  
  "source_text": "■■■■■, ■■■■■ ■■■■■ ■■■■■■■■■■■■■?",  
  "translated_text": "Hello, how are you?",  
  "source_language": "ta",  
  "target_language": "en",  
  "model_id": "translation_candidate"  
}
```

8.4 API error conventions

- 400: invalid request, unsupported language, invalid audio format.
- 413: upload or text exceeds configured limit.

- 422: schema validation failure.
- 503: model unavailable or not installed.
- 504: inference timeout.
- 500: unexpected internal failure; return a request ID and safe error detail, not stack traces.

9. Frontend Product Design

Use React + Vite for a responsive Voice Studio. Keep the UI straightforward: the user should be able to select a task, select a language, provide input, run inference, and review the result.

9.1 Main screens

- **Text to Speech:** text area, language dropdown, voice dropdown, Generate button, audio player, download.
- **Speech to Text:** upload or record, language dropdown, Transcribe button, transcript area, Copy and Translate actions.
- **Translation:** source/target language selectors, input and output panels, swap-language control where valid, Speak output button.
- **Voice Library:** list enabled default voices; future cloning flow includes consent and deletion.
- **Settings:** backend status, enabled models, local model paths, file-size limits, privacy/retention controls.

9.2 UI states

- Idle, uploading, processing, success, empty result, unsupported language, model missing, timeout, and failed decoding.
- Disable duplicate submissions while a request is running.
- Do not show a success state before the backend confirms completion.
- Display clear warnings if a feature is experimental or a language has not passed quality checks.

9.3 Suggested frontend stack

- React + Vite; TypeScript recommended for request/response types.
- Fetch or Axios for API calls; one shared API client with timeout/error handling.
- Use browser audio playback for supported returned formats.
- Avoid storing uploaded audio or generated speech in persistent browser storage by default.

10. Data, Privacy, Security, and Consent

Speech can contain names, health information, financial details, and other sensitive content. Local processing reduces third-party exposure but does not automatically make an application secure. Temporary files, logs, model endpoints, and backups still require controls.

10.1 Minimum controls for the local MVP

- Bind the development API to localhost by default; do not expose it to a public network without authentication and security review.
- Restrict accepted file extensions and verify the decoded media format rather than trusting the filename.
- Set maximum upload size, duration, text length, and inference timeout.
- Use generated temporary filenames; never build filesystem paths directly from user-supplied names.
- Delete temporary audio after processing and provide a clear retention setting if persistence is added.
- Do not log audio bytes, reference recordings, API secrets, or full user text by default.
- Keep model downloads and Python dependencies pinned/reviewed where feasible.
- Provide clear errors without exposing internal paths or stack traces to the UI.

10.2 Voice consent

- Require confirmation that the user owns or has permission to use the reference voice.
- Explain what the recording is used for and whether it is retained.
- Provide a delete-voice-profile option if voice profiles are stored.
- Do not use a voice sample to train or improve a model beyond the stated purpose without separate permission.

10.3 Local versus hosted deployment

For the first MVP, run the backend and models locally. If a hosted version is later offered, add authentication, rate limits, encryption in transit, secure file storage, deletion policies, abuse monitoring, and an explicit privacy notice before accepting public users.

11. Repository Structure and Implementation Conventions

```
xyraspeech/  
  backend/  
    app/  
      main.py  
      config.py  
      api/  
        routes_health.py  
        routes_tts.py  
        routes_stt.py  
        routes_translate.py  
        routes_text_prepare.py  
      schemas/  
        tts.py  
        stt.py  
        translation.py  
        text_prepare.py  
      services/  
        language_router.py  
        tts_service.py  
        stt_service.py  
        translation_service.py  
        ollama_service.py  
        temp_files.py  
      models/  
        registry.py  
    tests/  
      test_health.py  
      test_language_routing.py  
      test_api_validation.py  
      test_translation_contract.py  
  requirements.txt  
  .env.example  
  frontend/  
    src/  
      components/  
      pages/  
      services/api.ts  
      types/  
    package.json  
  models/                                # local model cache or documented external paths  
  samples/  
    tamil_test_sentences.txt  
    english_test_sentences.txt  
  scripts/  
    smoke_test.py  
  README.md  
  LICENSES.md
```

11.1 Engineering conventions

- Keep inference code out of route handlers; route handlers should validate and call services.
- Load configuration from environment variables with safe local defaults.
- Use type hints and Pydantic schemas for request and response contracts.
- Use structured logs with request IDs and elapsed time, but avoid sensitive payload logging.
- Keep model files outside Git unless the model license and repository size make inclusion appropriate.
- Document each model's source, version, checksum if feasible, license, languages, and verified tasks.

12. Three-Day MVP Execution Plan

A three-day plan is feasible only if pretrained models are available for the target machine and installation issues are manageable. The plan is a focused MVP schedule, not a guarantee that every candidate model will run on every Mac.

Day	Work	Deliverable	Exit check
Day 1 — Model validation and backend skeleton	Confirm Mac chip/RAM/disk; create virtual environment; test one Tamil TTS, one STT model, one translation path; create FastAPI health and schemas.	Model decision record + API skeleton	Each chosen model completes one local sample or has a documented blocker.
Day 2 — Integrate core features	Implement TTS/STT/translation services; add validation, temp-file cleanup, language router, errors, and smoke tests.	Working backend endpoints	Tamil TTS, Tamil/English STT, and Tamil ↔ English translation pass basic tests.
Day 3 — UI, QA, and documentation	Build React Voice Studio; connect API; test audio playback/download; add README, setup instructions, known limitations, and demo script.	Local end-to-end MVP	A user can complete each core journey from the browser.

Recommended scope discipline

- Do not train a model from scratch during the MVP.
- Do not implement voice cloning before the standard Tamil TTS path works.
- If one model is too large for the Mac, test a smaller supported variant or defer that task; do not silently switch languages.
- Treat model downloads and dependency compilation as schedule risks; validate these before UI polish.
- Reserve time for at least one clean installation and end-to-end demo.

12.1 Suggested day-by-day acceptance demo

Day 1: run a short Tamil sentence through the selected TTS model and verify the generated file plays. Transcribe a short Tamil sample. Translate one Tamil sentence into English. Day 2: repeat these through the API, including invalid inputs and model-not-ready cases. Day 3: complete all three workflows from the browser and demonstrate download/copy actions.

13. Testing and Acceptance Criteria

Testing must distinguish technical success (endpoint returns audio/text) from language quality (the result is intelligible and preserves meaning).

13.1 Functional test matrix

Test	Input	Expected behavior
Tamil TTS	Short Tamil-script sentence	Playable audio; Tamil is not translated.
English TTS	Short English sentence	Playable English audio.
Tamil STT	Clear Tamil recording	Tamil-script transcript, with reasonable punctuation if model supports it.
English STT	Clear English recording	English transcript.
Tamil → English	Tamil sentence	English translation; source text retained.
English → Tamil	English sentence	Tamil-script translation; meaning retained.
Unsupported language	Language not enabled	Clear 400/422 response; no silent fallback.
Missing model	Request to disabled model	Clear 503 with setup guidance.
Oversized upload	File over configured limit	413 response; no inference.
Ollama unavailable	Enhanced text mode	Clear error or configured fallback; core speech features still work.
Raw text mode	Text with spacing/punctuation	Exact same text returned without Ollama.
Temporary file cleanup	Any STT request	Temporary input removed after completion/failure.

13.2 Tamil language quality set

- Greeting and everyday conversational sentences.
- Formal written Tamil.
- Questions, exclamations, and long sentences.
- Sentences with numbers, dates, prices, and abbreviations.
- Place names and proper nouns.
- Tamil-English mixed technical terms.
- A quiet recording and a mildly noisy recording.

13.3 Quality scoring

For STT, compare output with a human-checked reference transcript and track word/character error rate where the script-aware evaluation is reliable. For translation, have a fluent Tamil/English reviewer assess meaning preservation and key details. For TTS, use human listening checks for intelligibility, pronunciation, pauses, and naturalness. Do not claim a quality score until it has been measured on a documented test set.

13.4 MVP definition of done

- The documented local setup works on the target Mac.
- All P0 endpoints return the documented response shapes.
- Tamil and English samples can be processed in the intended direction.
- The frontend can play generated audio and display/copy text results.
- Input validation, model-unavailable behavior, and temporary-file cleanup are tested.

- README lists exact commands, model requirements, limitations, and license notes.

14. Local Setup and Operations

Exact model installation commands depend on the selected model and Mac hardware. Keep model-specific setup in separate instructions rather than combining incompatible dependency stacks into one requirements file.

14.1 General backend setup

```
# From the project root
cd backend
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# Copy configuration template and adjust paths
cp .env.example .env

# Start the API
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

14.2 General frontend setup

```
cd frontend
npm install
npm run dev
```

14.3 Operational checklist

- Check /health and /health/models before testing a feature.
- Keep the API bound to localhost for local development.
- Check available RAM and disk space before downloading large models.
- Record model version and installation date in the model registry.
- Use a small known sample to verify each model after updates.
- If inference fails, check logs for model loading, audio decoding, dependency, and memory errors.
- Avoid updating all model and Python dependencies at once; change one component and rerun tests.

Note

These are generic project commands. The actual requirements.txt, model download commands, and model paths must be written after the candidate models have been selected and tested on the target Mac.

15. Risks, Limitations, and Mitigations

Risk	Impact	Mitigation
Tamil TTS quality is weak	Unnatural pronunciation or poor intelligibility	Benchmark multiple Tamil-capable models; use a fixed Tamil test set; do not promise unsupported accents.
Voice-cloning model does not support Tamil	Cloned voice may speak Tamil poorly	Choose a Tamil-capable base TTS; treat cloning as experimental until validated.
Mac memory is insufficient	Slow inference, crashes, or model load failure	Use smaller/quantized models where supported; load lazily; avoid simultaneous large models.
Dependency conflicts	Installation failures	Use a venv, pin tested versions, isolate model integrations.
Mixed-language recognition errors	Incorrect transcript for Tamil-English speech	Label as experimental; add code-switching test data and user language override.
Translation changes names/numbers	Meaning or detail loss	Test proper nouns, numerals, and technical terms; allow review before TTS.
Model licensing ambiguity	Commercial-use risk	Review code, weights, dataset, and dependency licenses separately.
Voice misuse	Impersonation or privacy harm	Require authorization, restrict retention, add deletion and usage rules.
Local inference is slow	Poor user experience	Show progress, use timeouts, benchmark latency, consider optional later hosting.
Unrequested text changes	Meaning drift or accidental translation	Keep raw mode exact; separate translation from text preparation; test output.

The platform should prefer a clear limitation or error over a misleading result. For example, if Tamil is not supported by a selected voice model, the API must not silently generate English speech.

16. Commercial Readiness and Licensing Checklist

A local model can be free to download and still have restrictions on commercial use. Before using XyraSpeech in a paid product, review each component independently.

- Source-code license for the inference library or repository.
- Separate license and terms for the model weights/checkpoints.
- Dataset terms and any restrictions inherited from training data.
- Commercial use, redistribution, modification, and hosted-service permissions.
- Attribution or notice requirements.
- Restrictions on voice cloning, impersonation, or biometric use.
- License compatibility across Python packages, audio codecs, and frontend libraries.
- Privacy and consent requirements for user recordings.
- Terms for any optional external service, if one is introduced later.

Release gate

Do not advertise a model as commercially cleared until the relevant code, weights, data terms, and dependencies have been reviewed. Record the license URL, date reviewed, version, and decision in LICENSES.md.

17. Future Roadmap

Phase	Features	Exit condition
MVP	Tamil/English TTS, STT, Tamil ↔ English translation, Voice Studio, local API	End-to-end local demo and documented tests
Phase 2	Voice profiles, improved error handling, batch jobs, model benchmarking	Stable repeated use and measured quality
Phase 3	Consent-based Tamil voice cloning, speaker presets, supported style controls	Verified Tamil base model and consent workflow
Phase 4	Hindi, Telugu, Malayalam, Kannada and other Indian languages	Each language has validated models and test cases
Phase 5	Optional conversational voice assistant: STT → local LLM → TTS	Low enough latency and robust turn handling
Phase 6	Hosted API, user accounts, quotas, billing, team features	Security, privacy, licensing, cost, and operational review

Potential later enhancements

- Batch TTS for long documents and subtitle-aligned audio where supported.
- SSML or equivalent control only if the selected TTS engine supports it.
- Streaming speech output and partial STT transcripts if model/runtime supports them.
- Voice library with language, style, sample audio, and license/consent metadata.
- Evaluation dashboard for latency, model version, transcription error, and user-reported pronunciation issues.
- A plugin architecture for adding new models without rewriting the API.

18. Appendix: Sample Requests and Demo Script

18.1 Tamil test text

ஹெலோ! ஹவு அரே யூ துடே? ஐ கான் ஹெல்ப் யூ வித் யூர் வர்க்.
ஹெலோ! ஹவு அரே யூ துடே? ஐ கான் ஹெல்ப் யூ வித் யூர் வர்க்.

Use this as a basic smoke test only. It is not a comprehensive Tamil benchmark.

18.2 English test text

Hello! How are you today? I can help you with your work.

18.3 Example end-to-end demo

- Open the local Voice Studio and verify backend health.
- Select Tamil in Text to Speech, paste the Tamil test sentence, generate audio, play it, and download it.
- Upload a clear Tamil recording and transcribe it; confirm the transcript is displayed in Tamil script.
- Translate the transcript from Tamil to English and review the result.
- Select Speak Translation to generate English speech from the translated text.
- Run the English-to-Tamil translation path and verify the output is in Tamil script.
- Show the model status page and explain any features that are still experimental.

18.4 Suggested project README opening

XyraSpeech

A local-first, India-focused multilingual speech platform.

MVP goals:

- Tamil and Indian-English text-to-speech
- Tamil and English speech-to-text
- Tamil <-> English translation
- React + Vite Voice Studio
- FastAPI backend
- Optional local Ollama text preparation

Model support is task-specific. Check the model registry and LICENSES.md before enabling a language or using a model commercially.

Final summary

Build XyraSpeech around language-specific, replaceable model services. Prioritize reliable Tamil and English speech workflows before cloning or expanding the language catalog. Preserve the user's chosen language, make translation explicit, test on the target Mac, and document model quality and licensing honestly. This approach keeps the first release manageable while creating a foundation for a broader Indian-language speech platform.

Document status: Detailed project blueprint. Implementation and model validation remain engineering tasks.